AEOS · HARNESS IS THE PRODUCT ◆ STORM PRINTING V 27

◆

TRIA VOLUMINA · UNA LEX · EVIDENTIA AUT SILENTIUM

BUILD · TEST · PROVE · DOCUMENT · SHIP

10,000,000 AI ENGINEERING

Volume III The Arc Complete

THE EMBASSY, THE CONSULATE, HYSICIAN AN SELF-AUDIT V32) THE SYSTEM AT V32.0.0

AEOS

VOLUMEN III · TERTIA · 4,693 VERBA · ZERO DEPENDENTIAE

EDITIO PRINCEPS · AEOS PRESS · MMXXVI ◆ OPERA TERTIA

THIS IS NOT A GUIDE. THIS IS A RECEIPT.

A guide tells you what exists. A codex tells you what was built, in what order, with what proof, and how you can verify every claim with your own hands. Nothing in these pages is a proposal — every mechanism ships in the aeos repository, every property ends in the name of the test that enforces it, and the whole system runs offline with zero runtime dependencies.

Read it as an operator: run the proofs, then read the law.

```
pip install -e .        # zero runtime dependencies
python -m pytest        # 397 proofs, chaos storm included
aeos run-demo           # the whole OS, end to end
aeos storm              # the nuclear receipt: 9/9 scenarios survive
```

INDEX OPERUM

CONTENTS

10,000,000× AI ENGINEERING — VOLUME III

THE ARC COMPLETED

Distance, Co-Design, and the Federated Market — the System at v10.0.0

PREFACE: THE LAST THREE RUNGS

Volume II ended with a roadmap stated as engineering work with exit criteria: v8 (distance), v9 (co-architecting), v10 (the market). This volume documents that work done — the OS at v10.0.0: thirty-two modules, 161 tests, nineteen ADRs, and a closed arc that began three volumes ago with a single refusal: the harness is the product.

The pattern of the whole trilogy holds one last time. Each rung added reach without adding exceptions. v8 moved work across process and network boundaries — and every truth rule survived the trip. v9 let the machine propose architectures — as a ranked slate a human finishes, not a fait accompli. v10 opened the border to other organizations' capabilities — with one sentence enforced in code: **import is quarantine.**

And the fifth invariant — no self-modification without a spent human token — turned out to be the load-bearing wall for all three: remote workers and sandboxes extend what the system can touch, slates extend what it can imagine, federation extends where it can shop. None of it extends what it may become without a receipt.

Reproduce the volume:

```
python -m pytest                              # 161 proofs
aeos sponsor --scope "factory:install:X"      # issue authority
aeos console                                  # see it on the record
aeos federation-demo                          # watch the border work
```

PART I — DISTANCE (v8)

CHAPTER 1. THE TRUTH RULES TRAVEL

Distance is where harnesses go to die. The model moves behind an API, the tools move behind servers, validation moves into sandboxes — and quietly, guarantee by guarantee, the system becomes a distributed wish. v8's law: **distance changes the transport, never the truth rules.** Four mechanisms, one test suite.

`HTTPModelTransport` carries model calls over the wire and maps every failure onto the ADR-010 taxonomy — 503 and 429 are TRANSIENT, timeouts are TRANSIENT, overflow never retries — so the runbook's "repair the correct layer" survives the network intact (`test_503_maps_to_transient`, `test_timeout_maps_to_transient`).

Remote tool calls keep the posture that defines the OS: a result that crossed the wire is **still untrusted** — the constant is not a flag and geography does not launder it (`test_remote_tool_result_untrusted_over_the_wire`). A dead endpoint is a structured error, never an exception in the caller's frame (`test_dead_endpoint_is_structured_not_raised`).

CHAPTER 2. THE REMOTE COLLEAGUE

`WorkerServer` and `RemoteWorker` implement A2A-style delegation in its durable shape: POST a task, receive an envelope — typed, evidence-carrying, indistinguishable from a local handler's return. To the orchestrator's graph, a remote colleague is just another node; the graph's validation, serialization, and gates neither know nor care where the work happened (`test_delegation_roundtrips_envelope`).

The server's one rule of its own: broken remote handlers become error envelopes, never five-hundreds with stack traces (`test_broken_handler_becomes_error_not_stacktrace`). An agent that dies owe its supervisor a verdict, not a traceback — the same courtesy the defensive sandbox pays its parent in the next chapter.

CHAPTER 3. THE SANDBOX THAT CAN BE KILLED

v8's hardest property is almost embarrassingly mechanical: `run_isolated()` runs the factory's smoke validation in a **subprocess** with a wall-clock timeout from the parent, CPU and memory rlimits in the child where the OS provides them, and a child (`sandbox_runner`) that catches its own poison and writes a FAIL verdict instead of dying theatrically.

The tests are the specification. A candidate that cannot finish startup under a millisecond-scale budget is killed and reported — "sandbox exceeded its wall clock and was killed — a sandbox that cannot be killed is not a sandbox" (`test_hanging_candidate_is_killed`). A poisoned input — an unknown field smuggled into the contract — gets a written verdict naming the poison, exit code 1, no stack trace owed to anyone (`test_poisoned_input_child_writes_verdict`). The factory gained `isolation="process"` and drops into it without changing a single downstream expectation (`test_factory_isolation_process_mode`).

Sponsorship grew persistence in the same version: tokens live in JSONL, and spent stays spent across restarts (`test_spent_stays_spent_after_reload`) — authority that evaporates on reboot was never authority. And the console (`aeos console` + `aeos sponsor`) puts the whole ledger on one static page: proposals with their measured signatures, the audit trail of every token issued, spent, and refused, and the exact commands to act. Governance you cannot inspect is governance you cannot keep.

PART II — CO-DESIGN (v9)

CHAPTER 4. THE SLATE

v7's factory designs from one conservative template — safe, and quietly totalitarian: one philosophy, take it or leave it. Real architecture is a set of tradeoffs, and hiding tradeoffs inside a template is how systems end up over-privileged by default.

v9's `codesign.py` makes the conversation explicit. For each measured signature, three coherent philosophies:

- **conservative** — the proven v7 template;
- **minimal-privilege** — no write surface at all, escalates on any doubt;
- **reviewer-first** — the output contract grows an independent review artifact.

All three are contract-complete, sandbox-validated, and scored with least-privilege weighting — writes 0.5, evaluation strength 0.3, escalation off-ramps 0.2 — so **minimal-privilege ranks first** on any WRITE-shaped signature by construction (`test_least_privilege_ranks_first`). Then the human finishes the design by spending authority: the sponsorship scope includes the variant label (`codesign:triage-specialist:minimal-privilege`), so a token for the conservative variant cannot buy the minimal one (`test_wrong_variant_token_scope_refused`), and choosing twice is refused because one token is one power (`test_choice_with_token_and_scope`).

The honest limit, stated: the slate shares the template's ceiling. Three honest philosophies beat thirty-six perturbations of a template we only partially understand — but genuinely novel architectures remain human work. The machine explores the known; the human authors the new.

PART III — THE MARKET (v10)

CHAPTER 5. IMPORT IS QUARANTINE

The terminal layer answers the maximal trust question: what happens when capabilities arrive from other organizations — with reputation we cannot verify, from hashes we did not compute, for workloads we never measured?

One sentence, enforced in code and witnessed in the demo (`evidence/v10-federation-run.json`):

1. A foreign unit that passes its own hash enters **QUARANTINED** — it exists in our catalog, installed by no one.
2. Install while quarantined is refused **before any token check** — the demo holds a valid, correctly-scoped token and is still refused, because no sponsorship can outrank local validation (`test_quarantined_install_refused_even_with_token`). The refusal does not spend the token: quarantine is not the token's business.
3. The only road to TRUSTED is our sandbox, under our gates (`test_revalidate_promotes_and_install_succeeds`).
4. Tampered foreign artifacts are refused **at the border** — they never even become quarantined (`test_tampered_foreign_unit_never_enters`).
5. Export carries provenance — the same rule seen from the other side of the border (`test_export_bundle_carries_provenance`).

Web-of-trust and signed publishers are the right future addition — as a fast lane through quarantine, never as a substitute for local revalidation. Audit is how you learn; prevention is why you sleep.

CHAPTER 6. THE LEDGER, CLOSED

VOLUME	VERSION	TESTS	THE RUNG
I	v1.0	68	the kernel: contracts, gates, boundaries, governor
II	v7.0	131	platform → factory: adapters, runtime, catalog, economics, meta-loop
III	v10.0	161	distance, slates, the market

The five invariants, end to end: no envelope, no result; no authority without a boundary; no autonomy without reliability; no memory without validation; **no self-modification without a spent human token**. Every one is a test. Every test is one command.

CHAPTER 7. WHAT REMAINS (STILL HONEST)

v10 does not claim: real-fleet validation against production model providers at scale; a hardened MCP client against live servers; container-grade sandboxing (the in-process supervisor is the portable floor beneath it); a signed-publisher web of trust; multi-region deployment. Each is stated with exit criteria in the ADRs — because a roadmap you can't be wrong about is a poem, and this trilogy has had enough poetry.

The closing sentence of Volume I still stands, now with ten versions of receipts behind it: build systems that build systems; then systems that improve systems; then systems that discover what systems should exist — and stop, at every single door, to ask who is paying for the next one. The human moves upward. The machines execute downward. Between them: an operating system — built, tested, documented, governed, and now, federated.

CHAPTER 8. SECOND PRINTING ADDENDUM — LIVE MODELS BEHIND THE SEAM (V11)

The trilogy closed at v10 with one deliberate omission, listed in Chapter 7's honest gaps: real-model validation. v11 closes it the only way this project knows — as capability, tested, with an ADR.

One transport (`ChatCompletionsTransport`) speaks the OpenAI-compatible wire for every provider: **OpenRouter** (any frontier model behind one key), **Abacus AI RouteLLM** (OpenAI-compatible, ships with ChatLLM Teams), and OpenAI itself — env-resolved via `AEOS_PROVIDER`/`AEOS_MODEL`, so switching providers is an export, not a refactor. Its error contract maps the wire onto the ADR-010 taxonomy at the boundary: 429/5xx are TRANSIENT (retry, backoff, breaker), context-length 4xx is CONTEXT_OVERFLOW (never retry), other 4xx are PERMANENT (fail fast into repair). The reliability machinery built for a fake transport in v2 now governs real providers without changing a line.

Two rules carry the safety posture. **Keys are environmental, not architectural**: resolved at call time, refused loudly when absent ("live mode refuses to guess"), never stored, never logged — the event log's structural redaction (v1.1) was waiting for this day. And **spend is governed inline**: `MeteredAdapter` records real token usage from provider `usage` fields into the economics layer — live runs produce actual metered cost and leverage in the same evidence bundle shape, `mode: "live"` — and past `AEOS_MAX_COST` (default $2.00) the next call fails PERMANENT. A spend governor inside the seam: no graph, gate, or governor needs to know money exists.

The seam proof is the chapter's favorite test: a live-shaped adapter on a localhost wire drives the same reference graph to acceptance (`test_reference_run_accepts_live_adapter`) — acceptance identical, economics metered instead of estimated. The real-money smoke is opt-in (`AEOS_LIVE=1` + key) and skips with a message otherwise; `aeos live-check` shows the resolved config for exactly zero dollars. The default remains the deterministic engine. Tests stay free. Guarantees stay proven. The live path is a key away, not a rewrite away — which was ADR-001's promise, kept one more time.

◆

CHAPTER 9. THIRD PRINTING ADDENDUM — COMPANIONS (V12)

An OS that only trusts its own handlers is a walled garden; one that trusts outside agents is a sieve. v12 threads the needle: **companions** — external agents subcontracted as bounded nodes.

Pi (the coding agent the SSSF/fusion lineage runs on) joins as a builder backend, invoked exactly as the factory pattern does it: `pi -p --mode json --session-id`, prompt in argv, **stdin DEVNULL** — honoring their documented silent-hang lesson. Its JSONL event stream lands in the OS event log (redaction structural since v1.1). But the chapter's law is in the authorship of truth: **artifacts derive from the filesystem diff, never from the agent's self-report** — pi's final JSON is a claim; the snapshot diff is the fact; the gates re-check both. And the boundary is the harness's, not pi's promise: a companion writing outside its `writes:` globs is reverted and its phase dies — proven in the suite by a deliberately rogue fake (`test_roguish_pi_is_reverted_and_killed`), because the laws must hold especially when the worker is somebody else's program. A hanging companion dies at its wall clock. A missing binary fails structurally ("the OS will not guess").

DeerFlow (ByteDance's deep-research SuperAgent, headless `deerflow --json`) joins as a research backend under the v5 untrusted-source law: mined sources become findings at capped confidence (0.75 — enough to surface, never enough to canonize), the final answer is quarantined as unverified, and an unparseable or absent stream yields an **empty brief** — no sources, no fabrication. `aeos companions` reports detection and enablement for both.

Every test runs against fake executables: no installs, no keys, no spend. The companion layer is ADR-021 in one sentence: external agents receive authority on loan, never law on loan.

CHAPTER 10. FOURTH PRINTING ADDENDUM — THE TRIANGLE (V13)

The image that started this project's final arc showed a law written over an operating layer: **MORE CONTROL = LESS SPEED. CONTROL COSTS SPEED.** Every serious system engineer knows the trade is real; most systems handle it by accident — autonomy tuned here, gates tuned there, parallelism set by whoever configured last, and the argument about the trade conducted without measurement.

v13 makes the triangle a first-class citizen in three moves.

One dial, all the knobs. A `RunProfile` is a named stance that moves everything together. CONTROL: checkpoint-heavy (autonomy ceiling L3), strict gate set, process isolation, fusion on, workers serialized to 2 — verification density maximized, speed sacrificed knowingly. SPEED: eight workers, ceiling L5, lean gates, fast models — reach maximized. COST: a quarter-dollar budget, fusion off (fusion multiplies spend), cheap routing. BALANCED: everything this book has shipped since v1. Selecting is `aeos run-demo --profile control`.

Floors the dial cannot reach. No stance removes the core gates (`artifacts_exist`, `claims_are_backed`); none starts above L5 — L6 and L7 are earned by evidence, never selected from a menu; none touches write boundaries or the checkpoint-forever classes. The triangle bends; the law does not. The tests pin each sentence of that claim (`test_floor_gates_survive_every_stance`, `test_no_profile_starts_above_l5`).

The receipt. `measure_triangle()` computes the run's actual control (gate density, boundaries enforced, permission friction, isolation used), cost (metered dollars, tokens), and speed (tasks per second, waves, wall clock) from the event log — closing with a plain-language line: "bought verification (9 gate checks, 2 checkpoints) — paid with 4.12s and $0.4000." `aeos triangle` renders it. And the suite proves the thumbnail's law end-to-end: the control stance MEASURES more control than the speed stance on the same workload (`test_control_measures_more_control_than_speed`). Physics, receipted.

The operating layer this trilogy built was never about defeating the triangle. It is the mechanism that makes the trade deliberate — chosen per run, bounded by floors that no mood can move, and audited after the fact in the same evidence bundle as everything else.

CHAPTER 11. FIFTH PRINTING ADDENDUM — THE DIVIDEND (V14)

The finest memory systems of 2026 — ClaudeMem is the reference — proved a number worth naming: capture raw context, distill it into ~500-token typed observations, retrieve layers instead of transcripts, and long-running work consumes ~10x fewer tokens than re-reading history. The principle underneath deserves its name: **negative marginal token consumption** — each additional run should cost FEWER tokens than the no-memory baseline, because distilled recall replaces raw re-reading. Memory stops being a cost center and becomes an appreciating asset.

v14 ships the economics as law and ledger. **Distillation**: repeated episodic lessons compress into one semantic record per (task, outcome) — the tightest phrasing survives, validation counts attach as evidence (the canonical gate still applies), and the compression ratio is measured from actual record tokens, never asserted. The reference run now carries prior sessions' episodes (the cross-session thesis in miniature) and reports compression on every bundle.

Cache-stable JSON: `stable_prefix()` serializes the stable payload as canonical JSON — sorted keys, tight separators — so the same knowledge yields byte-identical prefixes across runs. That is precisely what provider prompt caches need to hit, and the tests pin it: same stable set, same bytes, different volatile tails riding last where they invalidate nothing before them. Structure is the cache key; determinism is the discount.

The ledger: per task-class, baseline (naive re-read) versus all-in (recall + amortized storage overhead). Negative marginal is a computed fact with a sign — the reference curve reads delta −1850: memory-inclusive recall at 150 tokens against a 2,000-token baseline. And the hard law closes the loop: **MEMORY MUST PAY RENT.** Every stored byte is token-weight carried into future assemblies; a canonical record never recalled is squatting, flagged by key and weight for the entropy path. Pollution was a confidence problem in v1; it is an economic crime in v14.

The trilogy's economics is now complete: leverage measured what attention buys (v4), the triangle measured what control costs (v13), and the dividend measures what memory returns. Outcomes per attention, dollars per guarantee, tokens per remembered lesson — all on the receipt.

CHAPTER 12. HORIZON PRINTING ADDENDUM — THE MARCH TO V22

Eight versions shipped as one march, each closing a named gap from the compliance audit, each earned with tests.

The Recall (v15). Retrieval learned to pay in layers: keys before snippets before transcripts, an FTS5 index over the store — sqlite3 is stdlib, and structure beat a dependency. A recall that stays in L0 is the dividend compounding; a snippet that does not fit the budget is skipped, and a layer that cannot be paid is not reported. The savings ship in every bundle.

The Fleet (v16). The orchestrator's every mutation became an event in an append-only file whose order is the truth and whose replay is byte-stable proof. The industry ships OTel; we shipped the same principle at the scale of one host, testable offline.

The Resume (v17). An AFK agent that restarts from zero was never AFK. A checkpoint written atomically after every task; a crash that leaves progress durable; a resume that runs only what remains — and the call log proves every side effect happened exactly once.

The Rubric (v18). The twelve leverage points stopped being a mindset and became a rubric: twelve rows, each bound to a mechanism, each PASS demanding evidence on disk. A vibe is not a leverage point.

The Standards (v19). Success is planned: the operator's law registered as [STD-n] ids, plans refused at the gate if they cite nothing or cite what is not registered. Standards became something a plan carries, not something memory half-remembers.

The Emissaries (v20). Aider and headless Claude joined Pi under the same law — contract in the prompt, artifacts verified against the filesystem, phantoms raised as errors, walls killing the hung, boundaries reverting the greedy. Every emissary, one law.

The Protocol (v21). The Model Context Protocol arrived as a stdlib client: JSON-RPC over a subprocess's stdio, walls on every read, and the federation law traveling with it — an imported tool is UNTRUSTED material no matter which server vouched for it.

The Horizon (v22). Cache hits became money you can read: usage blocks parsed into hit rates and effective tokens, the v14 prefixes finally pricing their own payoff. And the benchmark was written — against LangGraph's durability, Letta's memory, OTel's tracing, CrewAI's crews — with every "behind" named as a seam: distributed durability, MCP server mode, OTel exporters, eval suites. The horizon is honest: it moves only when you can say exactly where you stand.

The system at v22: forty-four modules, two hundred eighty-seven proofs, thirty-one decisions, zero dependencies — and a charter whose every compiled value ends in a test name.

◆

CHAPTER 13. COLONY PRINTING ADDENDUM — THE MIRROR, THE BRIDGES, THE COLONY (V23–V25)

The benchmark named the seams; three versions closed them.

The Mirror (v23). Eval suites arrived with one law intact: no model grades itself. Cases carry predicate judges and weights; a case that raises FAILS with the exception's name instead of crashing the suite; scores clamp. The self-eval points the mirror inward — six of the system's own laws exercised on real fixtures, from the standards gate to byte-stable prefixes. `aeos eval` returns nonzero when the system fails its own reflection: leverage you can put in a loop.

The Bridges (v24). The ecosystem speaks two dialects — MCP on the wire, OTel in the observability stack — and v24 speaks both. The MCP server is the client's mirror image, with a law the test can recite: the tool set is exactly the readers (audit, check, recall); no verb that writes is exposed, ever. The roundtrip proof — our v21 client talking to our v24 server — is the bridge verified from both banks. The OTel exporter translates the fleet stream into spans with content-addressed identities: byte-stable exports, FAILED mapped to ERROR, a file any collector can swallow.

The Colony (v25). The graph became explicit: nodes declare their requires and their conditions; waves execute in dependency order; context carries every output. Failure blocks dependents early; a skipped or failed dependency blocks its dependents too; a cycle ends BLOCKED — the colony cannot hang, because the no-progress break is the law. And a colony that skipped its declared graph reports DEGRADED, not OK: honesty extends to the orchestration itself.

The system at v25: forty-eight modules, three hundred seventeen proofs, thirty-four decisions, zero dependencies. The charter carries twenty-eight compiled values, each ending in a test name.

◆

CHAPTER 14. STORM PRINTING ADDENDUM — THE VAULT AND THE STORM (V26–V27)

The challenge was fair: three hundred green tests in a cozy sandbox are not proof. Proof is the system running where power cuts mid-write, disks fill, inputs are hostile, hardware is constrained, and the network does not exist at all.

The Vault (v26). The audit found the truth first: a single torn line in the memory store crashed the entire load; the flush rewrote the whole file non-atomically, so a kill at the wrong moment was total amnesia. The vault made every persistent write atomic — tmp file, fsync, rename, directory fsync — so a crash or a full disk never touches the original. Loads became tolerant: a torn line is data, quarantined to a `.torn` sidecar for forensics while the system continues. The workspace took a kernel-released lock: a killed run cannot strand it, because the operating system itself lets go of the lock when the holder dies. And the scanner never dials out — a scanner that checks the network by using it is a bug.

The Storm (v27). Chaos became a command. Eight scenarios, end to end, on the real system: SIGKILL the run three times at growing delays and recover; tear the persistent files mid-line like a real power cut; fill the disk at the worst moment and prove the prior evidence byte-intact; feed it empty, binary, and hostile intents; black out every socket and complete a full run anyway; finish under a 256MB address-space cap; refuse a second run on a locked workspace; kill the companion server mid-session. The storm found a real leak on its first pass — a broken pipe that should have failed closed — and fixed it. Then it joined the standard suite, so the receipts cannot rot: every `pytest` from now on is, among other things, a chaos test.

The system at v27: fifty modules, three hundred thirty-eight proofs, thirty-six decisions, zero dependencies — and one command, `aeos storm`, that answers the only question that matters about a system that claims to be resilient: prove it.

CHAPTER 15. SHIPYARD PRINTING ADDENDUM — THE SHIPYARD (V28)

The deployment review named what was missing between built and deployed; the shipyard closed the closable. The repo took its license and its pipeline: a CI matrix that must prove 351 tests — chaos storm included — on four Pythons before any pull request may merge. Long-lived state learned to declare its version: an `aeos_schema` header on memory, fleet stream, and checkpoints; legacy files load without ceremony; state written by a newer aeos fails closed with a name, never a guess. And storage learned to pay its keep: `aeos groom` migrates legacy state in place and archives all but the newest runs — nothing deleted, everything shelved, everything named in the receipt. The storm grew a flake policy: generous walls, and one disclosed retry where wall-clock sensitivity demands it — a receipt that needed a retry says so, because a quiet receipt is worth nothing.

◆

CHAPTER 16. SOAK PRINTING ADDENDUM — THE SOAK (V29)

Two proofs matured in this printing. The backup stopped being a hope and became an artifact with law: deterministic — sorted members, fixed metadata, a manifest stripped of clocks and paths so identical state yields byte-identical archives, provable by hash; verified — every member checked against its sha256 before a single byte touches the workspace, and any mismatch refuses the entire restore, because a corrupt backup must restore nothing, never something wrong. Caches are never carried: the recall index is rebuilt on restore, which is the proof that it is a cache. And the drill is permanent — backup, destroy, restore, run again is the ninth storm scenario, executed on every test run.

The soak made stability a receipt instead of an impression: N consecutive runs on one workspace, state accumulating, wall-clock mean and max, tokens and cost, memory growth, disk delta — the numbers a serious operator asks for before trusting a system with sustained work. The live soak carries the same law as everything else that touches money: opt-in only, hard dollar cap, metered by the run.

◆

CHAPTER 17. EMBASSY PRINTING ADDENDUM — THE EMBASSY (V30)

The last two ecosystem seams closed without surrendering the law that makes the system deployable anywhere: network is a room you enter on purpose. The MCP client learned the streamable-HTTP transport — one POST per request, JSON or event-stream replies, the same walls and the same fail-closed errors over the wire. The span exporter learned to push: OTLP over HTTP, retries only for the errors that deserve them, and a hostile wire answered with a receipt — attempts named, spans still safe on disk — never an exception, never a hang. Every bit of it is proven on the loopback range: disposable local servers playing the far end, so the proofs need no network that the machine does not already contain. The default path remains what it has been since the vault: provably, receiptably offline.

CHAPTER 18. CONSULATE PRINTING ADDENDUM — THE CONSULATE (V31)

The last door on the protocol floor opened on purpose: the consulate serves the same read-only tool law over HTTP that the stdio server serves over pipes — one `handle_request`, two transports, so the law cannot drift between them. The default bind is loopback; reaching the world is an explicit act, recorded in a command. The strictness paid for itself immediately: the consulate refused the client's notification because the client had been sending it with an id attached — a protocol violation the permissive test doubles had let pass. The client now sends true id-less notifications, and the wire roundtrip — our HTTP client against our HTTP consulate — is a test. And the build learned its last deployment lesson early: a metadata table placed carelessly swallowed a settings key, caught by validating the project file before shipping. The sdist and wheel pass twine's checks with the license aboard, and the name is verified free. The system is one account step from the shelf.

CHAPTER 19. PHYSICIAN PRINTING ADDENDUM — THE PHYSICIAN (V32)

The oldest claim in the charter became a check. Zero runtime dependencies was asserted in ADR-002 when there were fifteen modules and inspected by eye for thirty-one versions; now a physician parses every import the interpreter sees and classifies it — stdlib, internal, or violation — and any violation fails the command. The same pass examines the workspace (are the schemas current, are there torn sidecars, is a lock held, is the disk breathing, is retention due) and the repository (is the tree clean, do the tags exist), and renders a verdict with names: PASS, WARN, FAIL — a doctor that flatters is not a doctor, and a FAIL fails the command. Fittingly, the physician's first patients were its own bugs: it misread relative imports as violations and looked for the repository one floor down — both caught because the check ran against reality before it ran against the claim. The runbook was rewritten the same day, because a system that audits itself still owes its operator a current map.

APPENDIX — VOLUME III RECEIPTS

- **ADR-017** Distance: taxonomy-preserving transports, remote colleagues, killable sandboxes, persistent authority.
- **ADR-018** Co-design: three philosophies, least-privilege scoring, variant-scoped sponsorship.
- **ADR-019** Federation: import is quarantine; reputation is not a verdict.

Reproduce: `pip install -e . && python -m pytest` (161) · `aeos run-demo` (leverage 7.0) · `aeos sponsor` + `aeos factory-demo --token` (scoped install, others refused) · `aeos federation-demo` (quarantine → re-validate → install) · `aeos console` (the record).

— end of Volume III, and of the trilogy —

◆ END OF VOLUME III · 4,693 WORDS · GENERATED FROM THE AEOS REPOSITORY ◆